GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

Software Engineering 3.0 Era

Innovative Talent Cultivation Exploring Open Source and AI Integration

Kerry Zhu · Tongji University

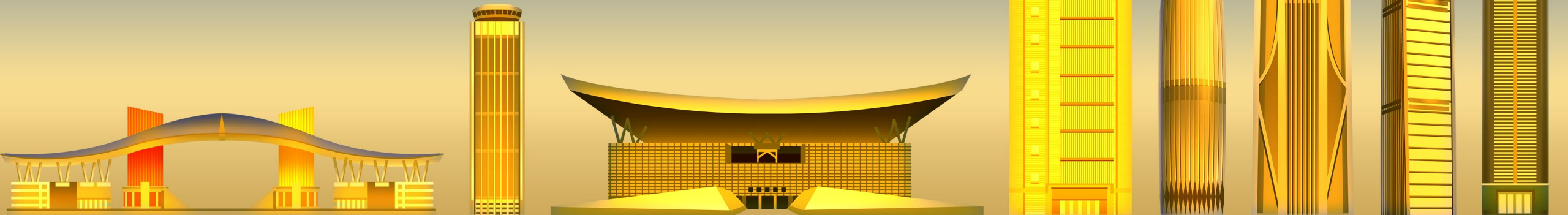
Author of “Software Engineering 3.0”



GOSIM

CONTENTS

Part 01	**The Impact of LLMs**	How AI is transforming software engineering education — impact & opportunity
Part 02	**Methodology**	Guarding critical thinking: counterfactual reasoning, debate, assessment
Part 03	**PBL Practices**	University cases, teacher roles, curriculum and talent frameworks
Part 04	**Vision & Outlook**	SE 3.0 as a long-term paradigm — a six-dimensional call to action



GOSIM

GOSIM Shenzhen 2026

0

IMPACT & OPPORTUNITY

The Impact of LLMs

1

From SE 1.0 to SE 3.0 — and what it means for education



Evolution: From SE 1.0 to SE 3.0

Three eras, three paradigms — each shifts what engineers must be good at.

SE 1.0

1968 · NATO Summit

Methodology → Structured Code

- Engineering standards
- Sequential development

SE 2.0

2001 · Agile Manifesto

SaaS, Cloud computing → Agile Iteration

- TDD, CI/CD, DevOps
- User feedback driven

SE 3.0

2023 · LLM Explosion

Model-Driven → Human-AI Collaboration

- AI-driven, model-first
- Self-adaptive agents

Core transition: 1.0 standardized 0→1 development · 2.0 rapid iteration & user collaboration · 3.0 deep fusion of AI and human intelligence.

SE 3.0 will persist for decades — not a brief transition.



Large Language Models: From “Can Think” to “Can Do”

Four rising capabilities are redefining what a software engineer does.

1 · Comprehension

Natural-language understanding of business needs — clearing the requirements bottleneck.

2 · Generation

One-stop code, design and test generation — from code completion to full project scaffolding.

3 · Collaboration

Formation of “human + AI” teams — small teams can now tackle large projects.

4 · Autonomy

Multi-agent systems that self-coordinate — self-optimization and self-evolution.

Critical Data

1M

lines of code from 3 people × 5 months

35%+

of coding tasks already automated

This is a revolution in “how we work,” not a “jobless future.”



Real Impact: Education Dilemma in the AI Age

Three shocks converge on a single, deeper problem.

Job Market Shock	Student Misconception	Education Lag
Age 22–25 employment ↓13% · Junior roles ↓67% (since 2022).	63% treat AI as the “primary” tool — over-reliance creates hollow competence.	Curriculum cannot keep up; evaluation standards become blurred.

“Will AI hollow out the junior developer pipeline?”

This is the ultimate anxiety of educators.

Is it a technical problem? NO **Employment problem? NO** **Teaching-paradigm problem? YES**

> **This is not a crisis, but an urgent signal for teaching models to upgrade.**



Opportunity: A Fundamental Dimension Shift

The bottleneck has moved from writing code to defining the problem.

Former: "How to write code?"	Now: "Why do it this way?"
Technical difficulty — AI can significantly reduce it.	Requirements definition — AI cannot replace critical thinking.

Learning efficiency ↑40%
24/7 instant feedback

Refocus on core abilities
From details → systems

Teaching revolution
Complex projects become feasible

Human Role Upgrade

Executor
↓
Orchestrator
↓
Architect

"AI lowers the implementation threshold, but raises the thinking threshold. No progress means falling back; progress is limitless."



GOSIM

GOSIM Shenzhen 2026

METHOGOLOGY

Guarding the Thinking Mind

Avoiding cognitive outsourcing through new learning methods



Avoiding the Cognitive Outsourcing Trap

63% of students treat AI merely as a “content generator” — copy, no thought (Beihang University).

✗ Cognitive Outsourcing (Harmful)	√ Cognitive Enhancement (Helpful)
Copy → no validation → passive	Understand → deep review → iterative improvement
Result: critical thinking decays.	Result: skill elevation, tool integration.

Four Protection Mechanisms

① Mandatory Pre-Thinking

Submit personal thoughts before using AI

② Screen-Recording Comparison

Analyze behavior with vs without AI

③ Reverse Analysis Report

Deconstruct AI code’s thinking process

④ Reflection Summary

Explicitly state “what I learned”

Teaching design determines whether AI enhances or replaces thinking.



Reverse Engineering Learning

From “passive acceptance” to “active questioning.”

Read	→	Question	→	Compare	→	Reconstruct	→	Reflect
Study AI’s final code		Why designed this way?		Compare solution paths		Reimplement yourself		Internalise knowledge

Core Value

- Fill gaps in AI’s missing “thinking process”
- Shift from passive to active thinking
- Break through the “standardized answer” mentality

Practical Tool · Codecademy AI Builder

- AI generates a complete project first
- System auto-decomposes concepts
- Personalized learning pathway
- “Learn Tab” deep explanations

> “Let you reverse-engineer how it was built, so you deeply understand and confidently modify it.” — Zoe Bachman, Codecademy



Ablation in AI-Era Software Engineering

Ablation = Remove one component. Hold all else fixed. Replay same task. Compare evidence.

A/B is broad; ablation is subtractive A/B

A general template for ablation experiments:

① **State a hypothesis,** e.g., legal-move observations reduce the illegal-move rate but increase token usage and cost.

② **Define the full configuration**, e.g., Model M + board + legal moves + play_move + error recovery + identical prompt.

③ **Construct the ablation configuration by disabling one variable**: board-only observation, or disabling compaction, or removing simulate_move / run_python / invoke_skill.

④ **Strictly control all other variables**: same model version, same prompt, same board position/task, same step limit, same temperature/seed, same token budget, same sandbox environment.

⑤ **Run the same task/benchmark**: e.g., run django__django-15368 in Part 2; run the same batch of chess games in Part 3. Preferably repeat multiple times to reduce randomness.

⑥ **Record metrics and evidence**: trajectories, artifacts, token usage, API costs, sandbox duration, metrics, etc.

⑦ **Conduct attribution analysis by comparing A/B differences** and explaining the benefits, costs, and limitations of the component. Grading focuses on the experiment and evidence

⑧ **Submit reproducible materials**, including token-usage-analysis.md, observation-experiment.md, trajectory JSON, patches, run commands, versions, and thresholds.



Counterfactual Reasoning for Critical Thinking

Forging causal reasoning — asking “what if?” instead of “what does it do?”

✗ **Old Question**

“What does the code do?” — passive description

√ **New Question**

“If we change this variable, what happens?” — active causal exploration

Three Teaching Examples

Database Performance	Distributed API	Microservice Reliability
If query logic shifts from a full-table scan to a hash index, how does performance change?	If the API switches from sync to an async queue, how does error rate change under peak load?	If the auth service stays healthy, can the checkout service SLO still be met?

Students learn system causality, architecture trade-offs and complex-system thinking. Counterfactual reasoning = core training for systems thinking.



Multi-Agent Debate: Judgment Through Collision

Multiple AI agents debate one issue; students observe trade-offs, not perfection.

Architect	“Recommend OAuth2 + JWT. Supports third-party login, strong extensibility.”
Security Officer	“JWT stored locally has XSS risk. How is token refresh and leakage handled?”
Devil’s Advocate	“OAuth2 is too complex and costly — overkill for small/medium apps?”
Innovation Officer	“Consider passwordless + biometric auth: secure AND better UX.”

Tool: Project Chimera (open-source reasoning engine)

Students Gain

- √ No perfect solution exists
- √ Practical trade-off training
- √ Reinforced questioning spirit

Goal: making rational decisions under uncertainty.



Assessment Redesign: Process over Results

What you assess, students prioritize. Assess thinking, cultivate thinking.

✗ Old model: “Does the code run?” YES → 100 | NO → deductions. Problem: cannot assess thinking quality; fails in the AI era.

Process 60%	Results 30%	Value-Add 10%
• Dialog quality • Reverse analysis report • Alternative solutions • Screen-recording comparison	• Code correctness • Engineering completeness	• Creative breakthroughs • Team collaboration

Practice · University of Maine screen-recording comparison

No-AI coding vs AI-assisted coding, compared & reflected on. Student discovery: “AI made me fast, but thinking got shallow” → self-awareness awakening.



Four-Layer Protection Against Outsourcing

Three questions decide it: explain the design · spot AI problems · modify with understanding.

Layer 1	**Mandatory Pre-Thinking**	Submit personal thoughts before using AI — prevents “thoughtless use.”
Layer 2	**Screen-Recording Comparison**	No-AI vs with-AI behavioral analysis — self-awareness awakening.
Layer 3	**Comparative Analysis Report**	Own version vs AI version compared deeply — forces questioning of AI output.
Layer 4	**Mandatory Reflection Summary**	“What did you learn? What is still unclear?” — internalize knowledge.

Core of the four-layer protection: let students see their own thinking process.



AI policy

1. AI tools are generally allowed for assignments and project reports unless a specific assignment states otherwise;
2. **Lecture highlights must not be AI-generated**; they must be written by the student themselves, based on their own understanding, questions, and associations after genuinely attending the lecture;
3. Regardless of how AI is used, **students are responsible for** accuracy, compliance, and understanding;
4. **Instructors verify understanding through quizzes on assignments/reports** (in-class quizzes, oral Q&A, online quizzes, code walkthroughs, project defenses, or individual questioning), and may require students to explain design decisions, code behavior, experimental results, and limitations.

This is not about banning AI; it is about prohibiting the use of AI to replace learning evidence that must be completed by humans. AI output is not authoritative, and the student must remain ultimately responsible.



GOSIM

GOSIM Shenzhen 2026

PRACTICE

PBL, University Cases & Talent

From theory to campus: real programmes, real outcomes



Case 1 · Zhejiang University of Technology

Knowledge graph + human-AI synergy three-stage model.

2026 National Higher Education Software Engineering Conference — Special Prize

Stage 1 · Cognitive Discovery

2–3 weeks

AI generates concepts → student understanding test → system identifies knowledge gaps.

Stage 2 · Deep Understanding

3–4 weeks

Counterfactual reasoning → AI real-time feedback → build causal relationship networks.

Stage 3 · Practical Application

4–6 weeks

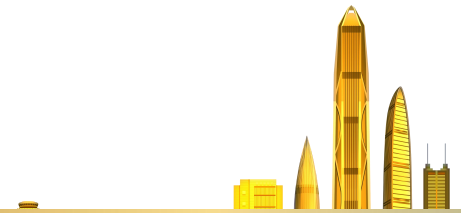
Real-world projects → multi-AI tool collaboration → open-source contribution practice.

↑40%
Knowledge Mastery

↑35%
Self-Learning

↑3×
Open-Source

Industry-education: Huawei & Tencent as internship bases · real projects as teaching material · GitHub contributions become a job-search advantage.



Case 2 · Tougu Platform: The 1+6 Model

Industry-education integration covering teach · learn · practice · evaluate · manage.

1 Platform (Tougu Cloud Experiment) + 6 Intelligent Functions → full-chain coverage

1 Question Bank Management	2 Code Evaluation	3 Learning Analytics
4 Personalized Recommendation	5 Teacher Dashboard	6 Enterprise Connection

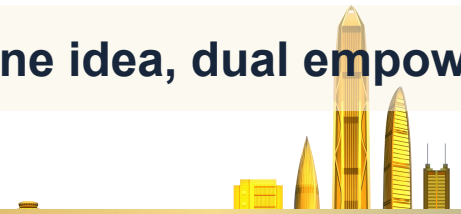
Hunan Normal University

- Personalized cultivation practice
- 500+ intervention warnings issued
- Significant grade improvement

Chongqing University

- First National Teaching Innovation Contest — 1st Prize
- “Operating Systems” full-chain synergy
- AI helps build problem-logic cognition

One idea, dual empowerment: students learn, enterprises get hire-ready talent.



Case 3 · Tongji “Jiqi” Teaching Agent

A vertical-domain agent built on an open-source ecosystem.

CivilGPT
Civil-engineering vertical LLM

+

Knowledge Graph
Structured civil-engineering knowledge

+

iS3 Digital-Physical Twin
Virtual-real fusion platform

Foundation Course	Innovation Course	Digital-Smart Practice
Adaptive self-assessment + AI explanation → reinforce weak areas.	Paper-standard advisor + literature reader → student evolves into researcher.	Configurable development + systems thinking → engineering & research capability.

> “Like an always-available tutor & assistant — freeing teachers from repetitive Q&A to focus on deep-thinking guidance.”

Open-source practice: LangChain & LLaMA frameworks · open-data utilization · student participation in open-source projects.



Teacher Transformation: Explainer → Designer

Teachers are not replaced — they evolve into “thinking guides.”

Traditional Role		SE 3.0 New Role
Lecturer	→	**Learning Designer**
Knowledge Authority	→	**Question Poser**
Answer Provider	→	**Thinking Guide**
Result Evaluator	→	**Process Companion**

Support

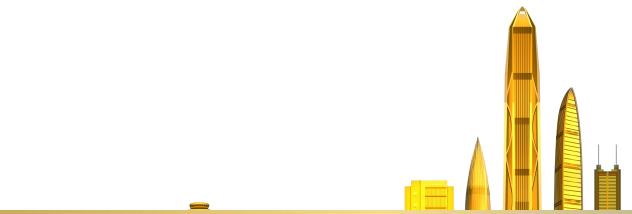
AI skill certification

Open-source community

Industry rotation

Evaluation tools & library

Design open-ended questions: ✗ “Write a login system” → √ “How do you balance security with user experience?”



New Foundation of Talent Competitiveness

Four mutually reinforcing dimensions of the SE 3.0 engineer.

1	**Critical Thinking**	Question AI output; ask “what if…?” counterfactuals; judge independently.
2	**Systems Thinking**	Make architecture decisions under multiple constraints; weigh trade-offs.
3	**AI Collaboration**	Express requirements precisely (prompts); iterate to improve AI outputs.
4	**Open-Source Contribution**	Submit high-quality PRs; participate in global community building.

All four dimensions are essential and mutually reinforcing — coding is now the foundation, not the summit.



Curriculum Reconstruction: Old vs New

Curriculum should run 1–2 years ahead, preparing talent for the future.

Dimension	Old Paradigm	New Paradigm
Teaching focus	Write code	+ Evaluate & architect code
Assignment focus	Code implementation	Critical analysis & reconstruction
Tech selection	Single language	+ Systems design + multi-ecosystem
AI role	Forbidden tool	Essential collaboration partner

Foundation · Year 1–2	Collaboration · Year 2–3	Practical · Year 3–4
AI-free PureCode training · deep CS fundamentals · manual debugging	Reverse-engineering learning · counterfactual reasoning · multi-agent debate	Real project internships · open-source contribution · ability integration

Key requirements: industry experts deeply involved in each course · annual curriculum updates · real-time alignment with enterprise talent needs.



PBL: Problem-Driven Learning Upgraded

From solving known problems to defining and solving real ones.

Previous PBL: solve known problems → **SE 3.0 PBL: define & solve real problems** → **Future: discover & define problems**

Create real context	Pose challenging problems	Self-directed exploration	Practical training	Solve & deliver	Reflect & optimise

Problem Design Elements

Authenticity
From real enterprise / social needs

Complexity
No simple AI shortcut

Openness
Multiple viable solution paths

Value
Real business / social value

Assessment & Reflection

- √ Process 55%
- √ Results 35%
- √ Reflection 10%

PBL is not just a learning method, but a learning ecosystem.



GOSIM

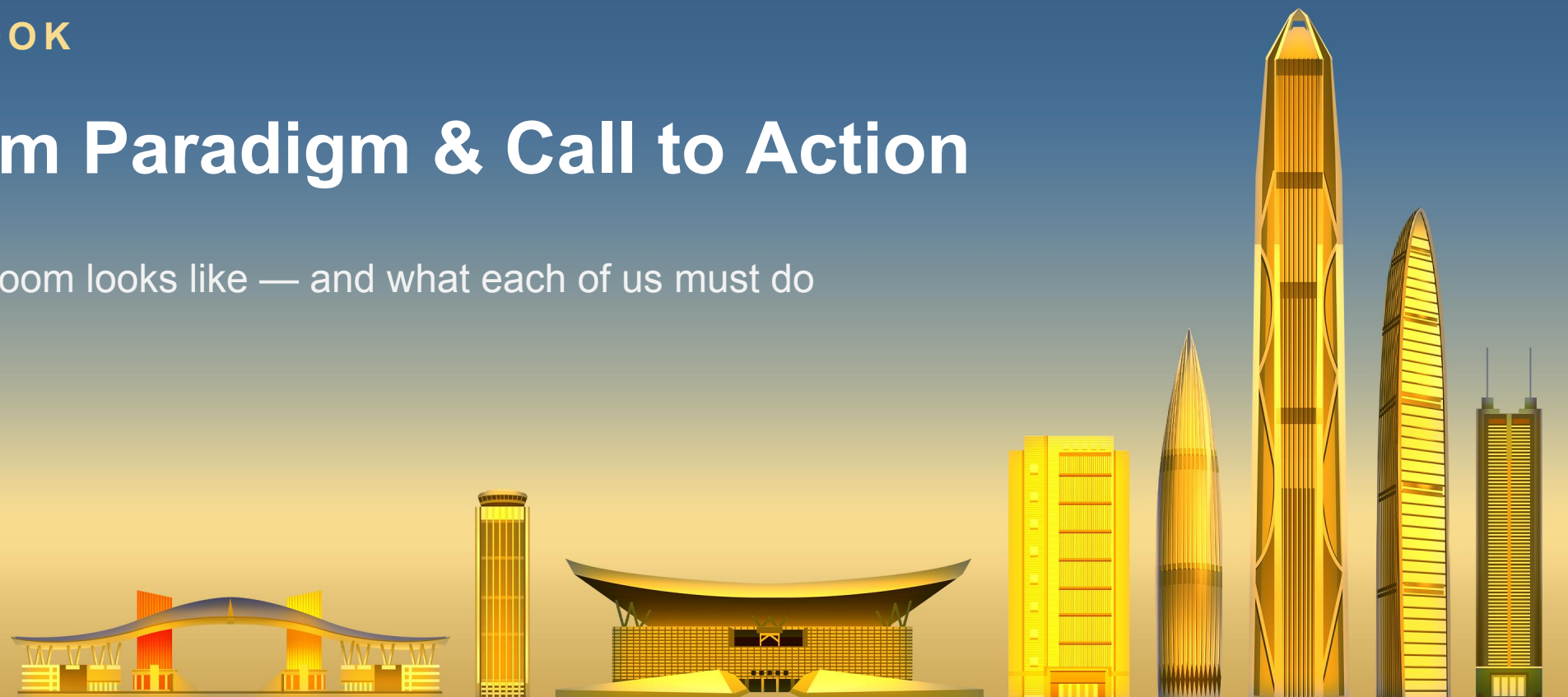
GOSIM Shenzhen 2026

04

VISION & OUTLOOK

A Long-Term Paradigm & Call to Action

What the future classroom looks like — and what each of us must do



SE 3.0: Paradigm & Call to Action

Not internal competition — a dimensional leap in capability.

Dimensional leap	“Why” over “how”	Decades-long
AI lowers the technical threshold but raises the critical-thinking and problem-definition threshold.	AI handles “how to do”; humans focus on “why” and better ways to do it.	SE 3.0 will persist for decades — a deep, long-term paradigm shift, not SE 4.0 next year.

Future Classroom

Students	Teachers	Evaluation	Teaching
Code porter → systems designer	Sole knowledge source → challenge designer	“Does code run?” → “What good questions?”	Output-driven → thinking-intensity-driven

Educators	Enterprises	Students	Policymakers
Transform the teaching paradigm; integrate open source.	Deepen industry-education fusion; share real projects.	Embrace change; master AI; ask better questions.	Drive systemic reform; modernize curriculum & evaluation.

SE 3.0 is not an endpoint, but a sustained practice platform — cultivating talent who can both master AI and define meaningful problems.



GOSIM

GOSIM Shenzhen 2026

Thanks!

Cultivating creative composite talents who master AI and define meaningful problems.

